AdriàEspinoza_BielManté

March 31, 2026

[1]:
```
%reload_ext autoreload
%autoreload 2
import pandas as pd
import warnings
warnings.filterwarnings('ignore')
import altair as alt
alt.renderers.enable('png')
from preprocessing import *
from final_views.q1_ratings_evolution import make_plot_q1
from final_views.q2_viewers_evolution import make_plot_q2
from final_views.q3_correlation import make_plot_q3
from final_views.q4_weekday_viewers import make_plot_q4
from final_views.q5_seasonal_pattern import make_plot_q5
```

This notebook describes the celaning, data expolarating steps justification of the chosen visualizations and initial visualizations considered

Authors: Adrià Espinoza & Biel Manté

1 Data processing and exploration

1.1 Data loading and new variables (cambiar)

We first load the data and perform the following steps:

1. Create a new variable, `day_aired`, from `original_air_date` (used to answer Q4).
2. Keep only the dataframe columns relevant to our analysis:
   - `season`
   - `number_in_season`
   - `number_in_series`
   - `original_air_date`
   - `day_aired`
   - `imdb_rating`
   - `us_viewers_in_millions`
   - `title`
3. Exclude columns that are not needed to answer our questions:
   - `id` (for our purposes, it is equivalent to `number_in_series`)
   - `image_url`
   - `imdb_votes`

- `original_air_year` (can be derived from `original_air_date`)
- `production_code`
- `video_url`
- `views`

4. Add a new binary column indicating whether an episode is the last one in its season. This variable was used in draft visualizations, but not in the final selected views.

[2]:
```
path = "../data/simpsons_episodes_raw.csv"
df = create_clean_dataframe(path)
```

1.2 Check Null values and data types

We perform basic descriptive analysis to study the null values our dataset contains, as well as the data types of its variables

[3]:
```
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 600 entries, 0 to 599
Data columns (total 9 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   season                  600 non-null    int64
 1   number_in_season        600 non-null    int64
 2   number_in_series        600 non-null    int64
 3   original_air_date       600 non-null    datetime64[ns]
 4   day_aired               600 non-null    object
 5   imdb_rating             597 non-null    float64
 6   us_viewers_in_millions  594 non-null    float64
 7   title                   600 non-null    object
 8   is_last_episode         600 non-null    int64
dtypes: datetime64[ns](1), float64(2), int64(4), object(2)
memory usage: 42.3+ KB
None
```

We see that the dataset contains a small number of missing values. `imdb_rating` has 3 missing entries, `us_viewers_in_millions` has 6 missing values. We will decide in future sections how to deal with them

1.3 Check Unique values

Episodes by season:

[4]:
```
counts_df = df.groupby("season").size().reset_index(name="episode_count").
 ↪sort_values("season")
print(counts_df)
```

```
    season  episode_count
0        1             13
1        2             22
```

```
2         3              24
3         4              22
4         5              22
5         6              25
6         7              25
7         8              25
8         9              25
9        10              23
10       11              22
11       12              21
12       13              22
13       14              22
14       15              22
15       16              21
16       17              22
17       18              22
18       19              20
19       20              21
20       21              23
21       22              22
22       23              22
23       24              22
24       25              22
25       26              22
26       27              22
27       28               4
```

Seasons with maximum number of episodes:

[5]:
```
counts_df[counts_df["episode_count"] == counts_df["episode_count"].max()]
```

[5]:
```
    season  episode_count
5        6             25
6        7             25
7        8             25
8        9             25
```

Seasons with minimum number of episodes:

[6]:
```
print(counts_df[counts_df["episode_count"] == counts_df["episode_count"].min()])
```

```
    season  episode_count
27      28              4
```

Number of epsiodes by airing day:

[7]:
```
print(df["day_aired"].value_counts())
```

```
day_aired
Sunday       506
```

```
Thursday      89
Tuesday        2
Wednesday      2
Friday         1
Name: count, dtype: int64
```

Our findings analyzing the unique values of the variables where it made sense to do so are: - There are 28 seasons, and the maximum number of episodes in a season is 25, which is consistent with typical TV season structures. Seasons 6 to 9 share this maximum number - Season 28 has the minimun number of episodes with only 4. This is extreamly different compared to the rest of the seasons - The `original_air_date` column contains 591 unique dates out of 600 episodes, meaning that 9 episodes share the same air date. - The `day_aired` variable has only 5 unique values and is highly unbalanced: most episodes aired on Sunday (506), followed by Thursday (89), while Tuesday and Wednesday appear only twice each and Friday only once.

1.4 Outlier Detection

An outlier detection analysis was performed using the IQR method on the numerical variables `imdb_rating` and `us_viewers_in_millions`. The results show a very small number of outliers in `imdb_rating` (2) and a slightly higher number in `us_viewers_in_millions` (9).

The 2 outliers for `imdb_rating` belong to very low ratings. This are: * The 11th episode of season 9 * The 22th episode of season 11

[8]:
```
outliers_imdb_rating = count_outliers_iqr(df, "imdb_rating")
lb = outliers_imdb_rating[2]
ub = outliers_imdb_rating[3]
outliers_imdb_rating[0][['season','original_air_date','day_aired','imdb_rating',
                         'us_viewers_in_millions']]
```

[8]:

	season	original_air_date	day_aired	imdb_rating	us_viewers_in_millions
71	9	1998-01-04	Sunday	5.1	8.90
193	23	2012-05-20	Sunday	4.5	4.82

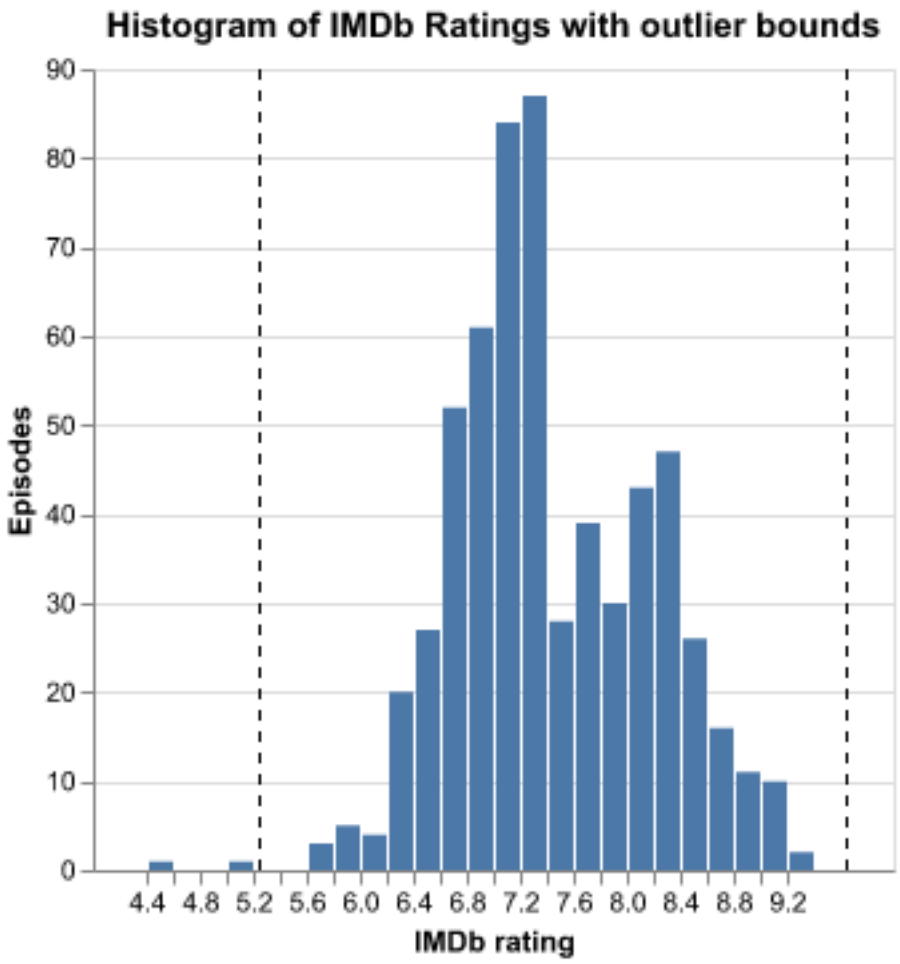
Since there aren't many outliers and the rating scale (0–10) isn't particularly large, keeping them won't significantly impact the distribution or future visualizations. So we'll keep them as they are.

Let's visualize the distribution of ratings and see where these outliers fall:

[9]:
```
hist = alt.Chart(df).mark_bar().encode(
    x=alt.X("imdb_rating:Q", bin=alt.Bin(maxbins=30), title="IMDb rating"),
    y=alt.Y("count():Q", title="Episodes")
)

bounds_df = pd.DataFrame({"bound": [lb, ub]})
bounds = alt.Chart(bounds_df).mark_rule(strokeDash=[4, 4]).encode(
    x="bound:Q"
 )
```

```
(hist + bounds).properties(title="Histogram of IMDb Ratings with outlier␣
↪bounds")
```

[9]:



We find 9 outliers on the variable us_viewers_in_millions, all of them belonging to early seasons of the show and larger audiences The Simpsons was transmitted right before the news programm in Fox News. Some meaningfull events happened on some of these days which could have made people want to watch the news more and indireclty boost the episodes audince (this is just observational and does not prove anyti).

Some of these special events were: * On March 25, 1990, a tragic arson attack at the Happy Land social club in the Bronx, New York City, killed 87 people * On April 29, 1990, the US space shuttle Discovery (STS-31) returned to Earth after deploying the Hubble Space Telescope. Major events included the LA Lakers defeating the Houston Rockets to set a record, Nirvana playing a legendary show in Washington, D.C., and Dan Quisenberry announcing his MLB retirement. * On December 3, 1992 The world's very first text was sent British engineer Neil Papworth sent the message "Merry Christmas" from his computer to the mobile phone of Vodafone director Richard Jarvis.

[10]:
```
outliers_viewers = count_outliers_iqr(df, "us_viewers_in_millions")
lb = outliers_viewers[2]
ub = outliers_viewers[3]
outliers_viewers[0][['season','original_air_date','day_aired',
                     'imdb_rating','us_viewers_in_millions']].
 ↪sort_values('original_air_date')
```

[10]:

	season	original_air_date	day_aired	imdb_rating	us_viewers_in_millions
26	1	1990-02-18	Sunday	7.9	27.6
245	1	1990-02-25	Sunday	7.7	28.0
217	1	1990-03-18	Sunday	7.5	33.5
0	1	1990-03-25	Sunday	7.4	30.3
218	1	1990-04-15	Sunday	7.8	31.2
1	1	1990-04-29	Sunday	8.3	30.4
2	2	1990-10-11	Thursday	8.2	33.6
220	2	1990-10-18	Thursday	8.3	29.9
24	4	1992-12-03	Thursday	8.5	28.6

1.5 Dealing with null values

From previous sections, we found 6 missing values in `us_viewers_in_millions`. Three of them come from season 28, which only has 4 episodes.

Keeping a season with just 4 episodes does not really help us identify trends compared with the other seasons, which contain many more episodes. It could also create confusion in the visualizations, for example making users think the dataset is incomplete. For this reason, we decided to remove episodes from season 28 and keep seasons 1-27, which are enough to answer the questions.

By removing season 28, we eliminated all missing values in `imdb_rating` and half of the missing values in `us_viewers_in_millions`.

[11]:
```
df_clean = df[df['season'] < 28]
```

To handle the 3 missing values that still remain in `us_viewers_in_millions`, we found a second dataset with a very similar structure to the provided one and matching values for `imdb_rating` and `us_viewers_in_millions`. We decided to plug in the missing values in our dataset using the equivalent entries from this second source.

This second source dataset can be found here: https://www.kaggle.com/datasets/jonbown/simpsons-episodes-2016?select=simpsons_episodes.csv

[12]:
```
dataset2 = pd.read_csv('../data/simpsons_episodes_second_source.csv')
```

[13]:
```
df_clean = df_clean.merge(dataset2[['season', 'number_in_season',␣
 ↪'us_viewers_in_millions']], on=['season', 'number_in_season'], how='left',␣
 ↪suffixes=('', '_from_ds2'))
df_clean['us_viewers_in_millions'] = df_clean['us_viewers_in_millions'].
 ↪fillna(df_clean['us_viewers_in_millions_from_ds2'])
df_clean.drop(columns=['us_viewers_in_millions_from_ds2'], inplace=True)
```

After this step, we have completed all the preprocessing steps and we can save our clean dataset

[14]:
```
df_clean.to_csv('../data/simpsons_episodes_cleaned.csv', index=False)
```

2 Questions justification

2.1 Q1 - How have the ratings evolved over time?

To answer this question, we chose a heatmap. This chart allows us to display the rating of every episode while making it easy to see how ratings evolve across seasons and within each season.

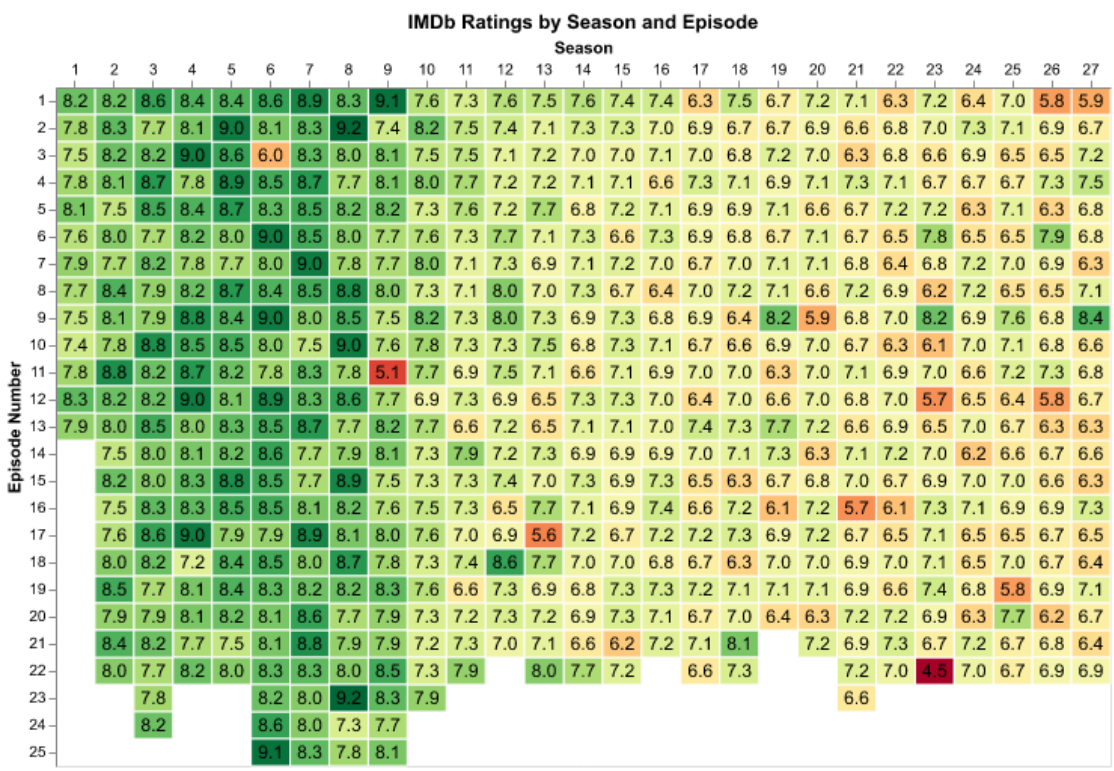
We use season number on the X-axis and episode number within season on the Y-axis. We also place the X-axis labels at the top to improve readability, so users can find the first episode of each season more quickly without having to make long visual jumps. In addition, we added white lines between tiles to create clear visual cells and separate ratings.

For color, we selected a red-yellow-green scale. This follows common visual intuition (red = worse, green = better) and makes interpretation faster. We ensured the palette was coloblind-friendly, and having a colorblind member in our group allowed us to test this. Using `scale = alt.Scale(range=['red', 'yellow', 'green'])` made low-rated episodes harder to distinguish, while switching to `scheme='redyellowgreen'` made distinctions much easier.

Finally, we removed the gradient legend in the final view because the color meaning is already intuitive. This saves space and makes the rating labels inside the cells easier to read.

[15]:
```
make_plot_q1()
```

[15]:



Episode Number	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27
1	8.2	8.2	8.6	8.4	8.4	8.6	8.9	8.3	9.1	7.6	7.3	7.6	7.5	7.6	7.4	7.4	6.3	7.5	6.7	7.2	7.1	6.3	7.2	6.4	7.0	5.8	5.9
2	7.8	8.3	7.7	8.1	9.0	8.1	8.3	9.2	7.4	8.2	7.5	7.4	7.1	7.3	7.3	7.0	6.9	6.7	6.7	6.9	6.6	6.8	7.0	7.3	7.1	6.9	6.7
3	7.5	8.2	8.2	9.0	8.6	6.0	8.3	8.0	8.1	7.5	7.5	7.1	7.2	7.0	7.0	7.1	7.0	6.8	7.2	7.0	6.3	6.8	6.6	6.9	6.5	6.5	7.2
4	7.8	8.1	8.7	7.8	8.9	8.5	8.7	7.7	8.1	8.0	7.7	7.2	7.2	7.1	7.1	6.6	7.3	7.1	6.9	7.1	7.3	7.1	6.7	6.7	6.7	7.3	7.5
5	8.1	7.5	8.5	8.4	8.7	8.3	8.5	8.2	8.2	7.3	7.6	7.2	7.7	6.8	7.2	7.1	6.9	6.9	7.1	6.6	6.7	7.2	7.2	6.3	7.1	6.3	6.8
6	7.6	8.0	7.7	8.2	8.0	9.0	8.5	8.0	7.7	7.6	7.3	7.7	7.1	7.3	6.6	7.3	6.9	6.8	6.7	7.1	6.7	6.5	7.8	6.5	6.5	7.9	6.8
7	7.9	7.7	8.2	7.8	7.7	8.0	9.0	7.8	7.7	8.0	7.1	7.3	6.9	7.1	7.2	7.0	6.7	7.0	7.1	7.1	6.8	6.4	6.8	7.2	7.0	6.9	6.3
8	7.7	8.4	7.9	8.2	8.7	8.4	8.5	8.8	8.0	7.3	7.1	8.0	7.0	7.3	6.7	6.4	7.0	7.2	7.1	6.6	7.2	6.9	6.2	7.2	6.5	6.5	7.1
9	7.5	8.1	7.9	8.8	8.4	9.0	8.0	8.5	7.5	8.2	7.3	8.0	7.3	6.9	7.3	6.8	6.9	6.4	8.2	5.9	6.8	7.0	8.2	6.9	7.6	6.8	8.4
10	7.4	7.8	8.8	8.5	8.5	8.0	7.5	9.0	7.6	7.8	7.3	7.3	7.5	6.8	7.3	7.1	6.7	6.6	6.9	7.0	6.7	6.3	6.1	7.0	7.1	6.8	6.6
11	7.8	8.8	8.2	8.7	8.2	7.8	8.3	7.8	5.1	7.7	6.9	7.5	7.1	6.6	7.1	6.9	7.0	7.0	6.3	7.0	7.1	6.9	7.0	6.6	7.2	7.3	6.8
12	8.3	8.2	8.2	9.0	8.1	8.9	8.3	8.6	7.7	6.9	7.3	6.9	6.5	7.3	7.3	7.0	6.4	7.0	6.6	7.0	6.8	7.0	5.7	6.5	6.4	5.8	6.7
13	7.9	8.0	8.5	8.0	8.3	8.5	8.7	7.7	8.2	7.7	6.6	7.2	6.5	7.1	7.1	7.0	7.4	7.3	7.7	7.2	6.6	6.9	6.5	7.0	6.7	6.3	6.3
14		7.5	8.0	8.1	8.2	8.6	7.7	7.9	8.1	7.3	7.9	7.2	7.3	6.9	6.9	6.9	7.0	7.1	7.3	6.3	7.1	7.2	7.0	6.2	6.6	6.7	6.6
15		8.2	8.0	8.3	8.8	8.5	7.7	8.9	7.5	7.3	7.3	7.4	7.0	7.3	6.9	7.3	6.5	6.3	6.7	6.8	7.0	6.7	6.9	7.0	7.0	6.6	6.3
16		7.5	8.3	8.3	8.5	8.5	8.1	8.2	7.6	7.5	7.3	6.5	7.7	7.1	6.9	7.4	6.6	7.2	6.1	7.2	5.7	6.1	7.3	7.1	6.9	6.9	7.3
17		7.6	8.6	9.0	7.9	7.9	8.9	8.1	8.0	7.6	7.0	6.9	5.6	7.2	6.7	7.2	7.2	7.3	6.9	7.2	6.7	6.5	7.1	6.5	6.5	6.7	6.5
18		8.0	8.2	7.2	8.4	8.5	8.0	8.7	7.8	7.3	7.4	8.6	7.7	7.0	7.0	6.8	6.7	6.3	7.0	7.0	6.9	7.0	7.1	6.5	7.0	6.7	6.4
19		8.5	7.7	8.1	8.4	8.3	8.2	8.2	8.3	7.6	6.6	7.3	6.9	6.8	7.3	7.3	7.2	7.1	7.1	7.1	6.9	6.6	7.4	6.8	5.8	6.9	7.1
20		7.9	7.9	8.1	8.2	8.1	8.6	7.7	7.9	7.3	7.2	7.3	7.2	6.9	7.3	7.1	6.7	7.0	6.4	6.3	7.2	7.2	6.9	6.3	7.7	6.2	6.7
21		8.4	8.2	7.7	7.5	8.1	8.8	7.9	7.9	7.2	7.3	7.0	7.1	6.6	6.2	7.2	7.1	8.1		7.2	6.9	7.3	6.7	7.2	6.7	6.8	6.4
22		8.0	7.7	8.2	8.0	8.3	8.3	8.0	8.5	7.3	7.9		8.0	7.7	7.2		6.6	7.3			7.2	7.0	4.5	7.0	6.7	6.9	6.9
23			7.8			8.2	8.0	9.2	8.3	7.9											6.6						
24			8.2			8.6	8.0	7.3	7.7																		
25						9.1	8.3	7.8	8.1																		

2.2 Q2 - How have the viewers evolved over time?

To show how viewership evolved over time, we initially thought a line chart would be the best idea. Using time on the X-axis and the number of viewers (in millions) on the Y-axis, along with season separators, makes it easy to track the show's growth and link specific peaks to their respective seasons.

Plotting all 596 episodes created a problem: the chart became too wide for the window, and the individual lines were so close together that the visualization looked cluttered. To fix this, we decided to plot the seasonal average as a solid line and display individual episodes as points. We lowered the opacity of the points to avoid oversaturating the plot while still havinng a way to see the "spread" of the data.

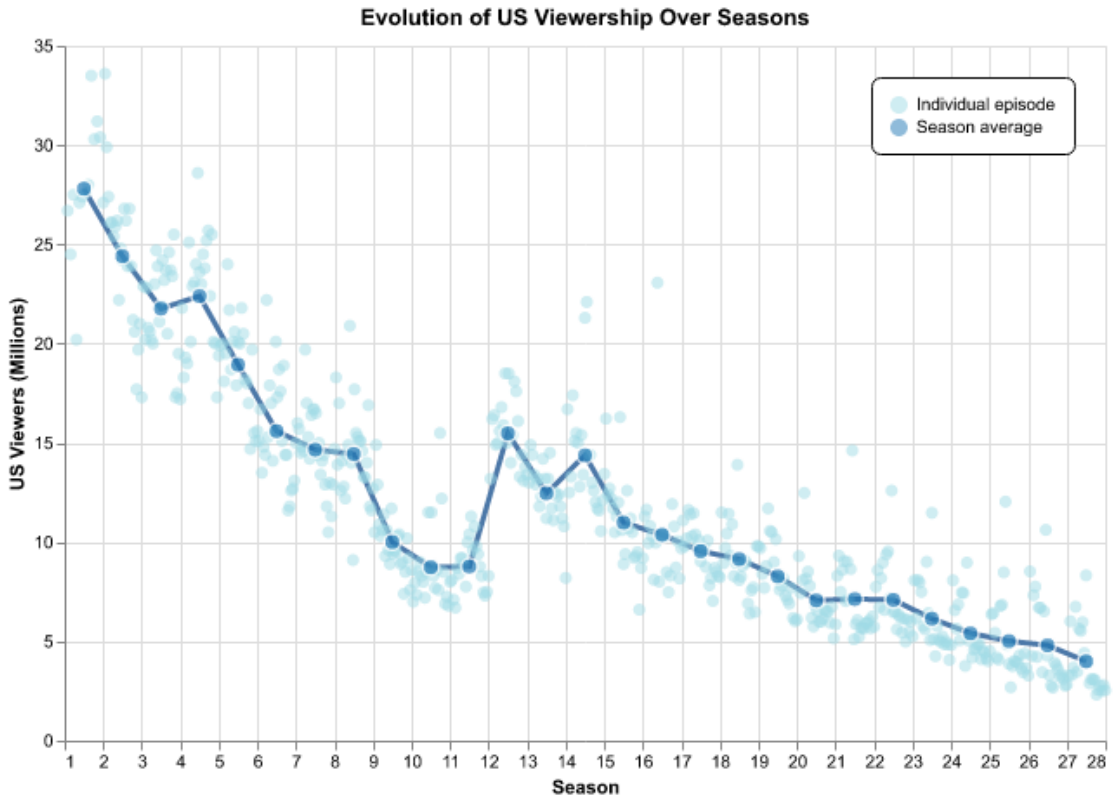
Even though we lose some detail by focusing on the mean, it highlights the overall trend accurately without being misleading. We kept the individual points so that the full detail for every season is still represented.

For the colors, we chose a strong blue for the seasonal average and a lighter cyan for the individual episodes. We used related colors to show that these measures are connected but distinct. The meaning of these colors is visible on the plot legend.

Moving forward, we'll stick to this logic for all our charts: bold blue for aggregated data and lighter tones for episode-specific values.

[16]:
```
make_plot_q2().properties(width=600, height=400)
```

[16]:



2.3 Q3 - Is there a correlation between the gradings and the viewers?

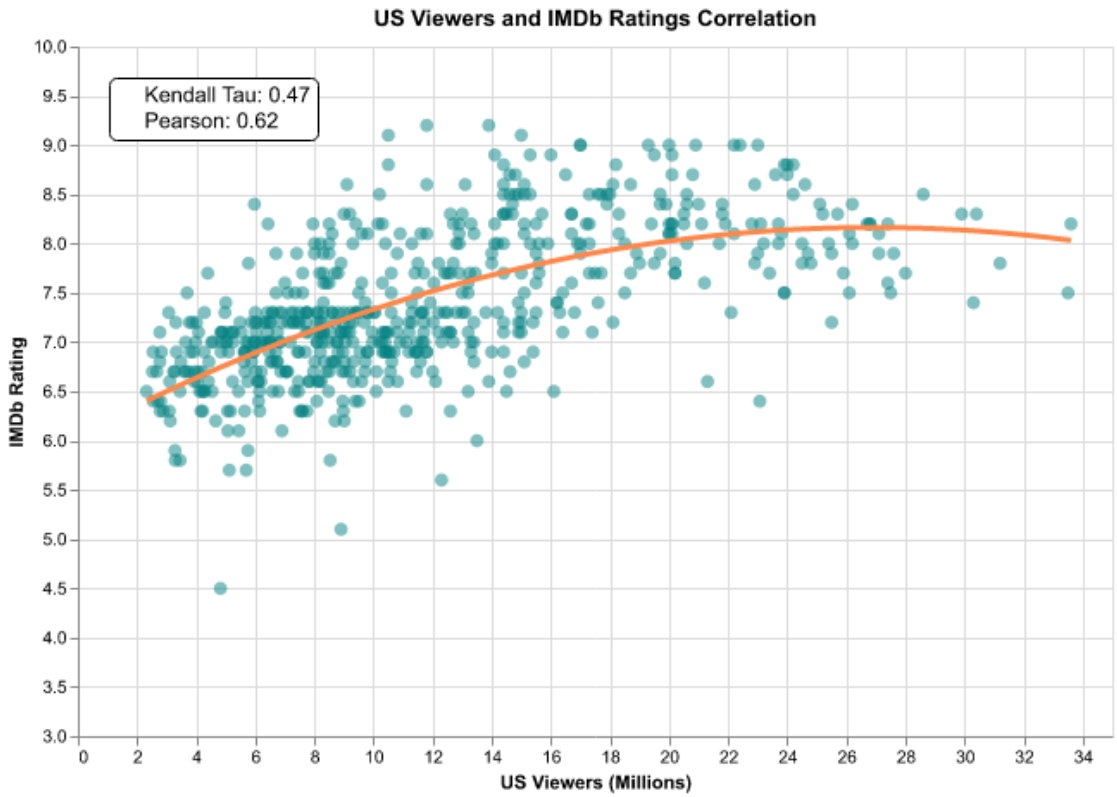
To answer this question, we chose a scatter plot as the main visualization for correlation. It is the clearest way to show how ratings and viewers move together episode by episode. We also added two labels with correlation values: Pearson and Kendall's tau. Pearson captures linear association, while Kendall's tau is useful when Pearson assumptions like normality or constant variance do not hold (like our EDA and previous plots suggest). They indicate a moderate relationship.

We also considered a dual-axis line chart to include the time dimension, but we decided against it. The scatter plot and trend line suggest a quadratic pattern, meaning viewers and ratings do not increase at the same rate. In that situation, a dual-axis chart can become misleading because interpretation depends too much on the chosen scales. Lines may look close or far apart just by changing axis limits, which can distort the message and bias interpretation. With the scatter plot, we can keep all individual points and still read the overall pattern clearly.

For the color, we decided to use one that was not used before, to reinforce the uniqueness of each dot representing the pair viewer-rating.

[17]:
```
make_plot_q3().properties(width=600, height=400)
```

[17]:



2.4 Q4 - Are the number of viewers for the episodes related to the weekday they were aired?

The first thing we decided to adress this question was that we should focus it to only study the days with a reasonable number of episodes, so we restricted it to Sunday and Thursday.

Earlier visualization tests suggested that Thursday episodes had more viewership, but we realized that all the Thursday episodes belonged to seasons 2 through 5. Since we already know the show had much higher viewership in its earlier years, concluding that Thursday is a "better" day could be misleading since the difference could come from the season, not the day of the week.

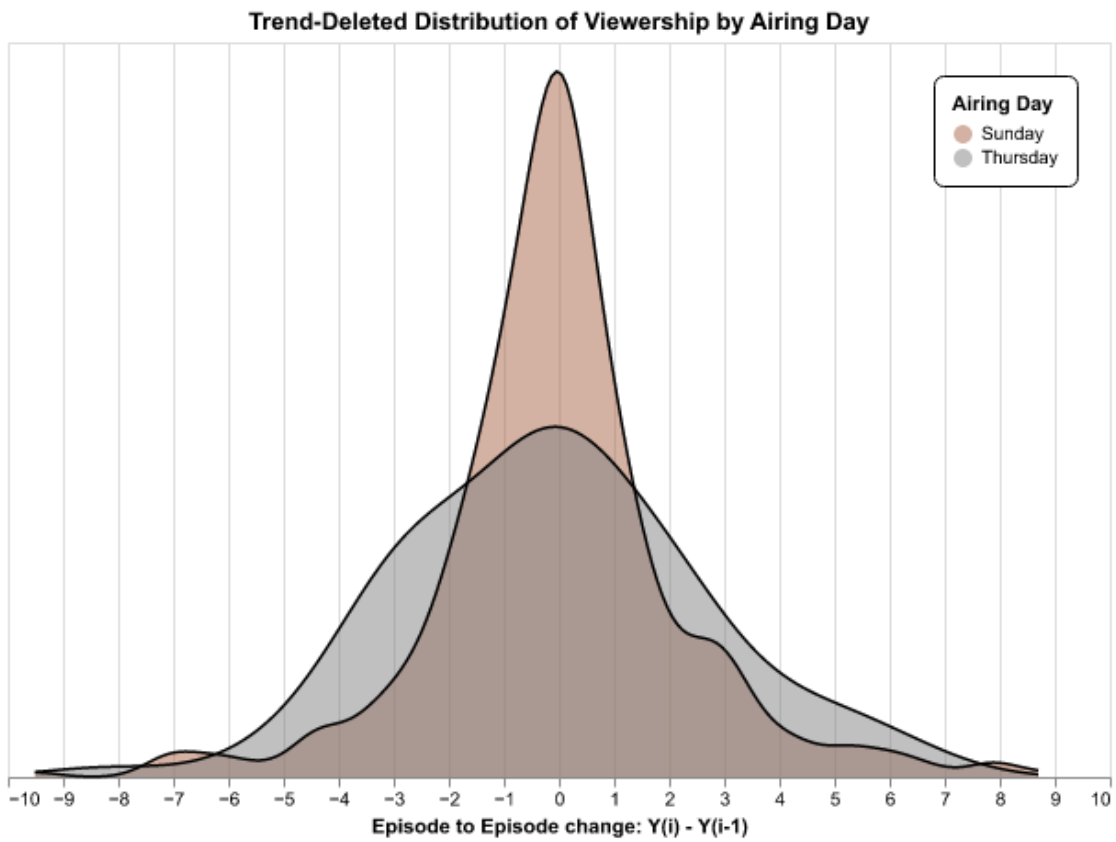
To address this, we transformed the viewership data using differentiation to remove the trend, a common technique for time series data. We then compared these detrended values using overlapping density plots to see the actual impact of the airing day, to fins out that the distributions were not so different, sharing a large area of the distribution.

We also removed a few extreme values from the visualization because they stretched the x-axis without adding useful information.

Since this plot encodes different information, we chose purple and pink for the weekdays to keep them distinct from the blue viewership tones and the red-yellow-green ratings used in other plots. We also deleted the y-axis since it does not contribute to understanding the distribuition better.

[18]:
```
make_plot_q4().properties(width=600, height=400)
```

[18]:



2.5 Q5 - Do the seasons number of viewers present any relevant pattern?

We approached this question by looking for patterns within individual seasons, for example, checking if premieres had more viewers than mid-season episodes.

Our initial idea was to create separate views for each season. However, the high number of seasons made this approach space-inefficient and difficult to compare them side-by-side.

We wanted a way to be able to represent seaons with different lenghts in a shared X-axis, so we introduced a new variable that normalized the episode number by the number of episodes of the season. Values close to 0 belong to early espidoes and the ones close to 100 to the later ones. In the Y-axis we displayed the number of viewers.

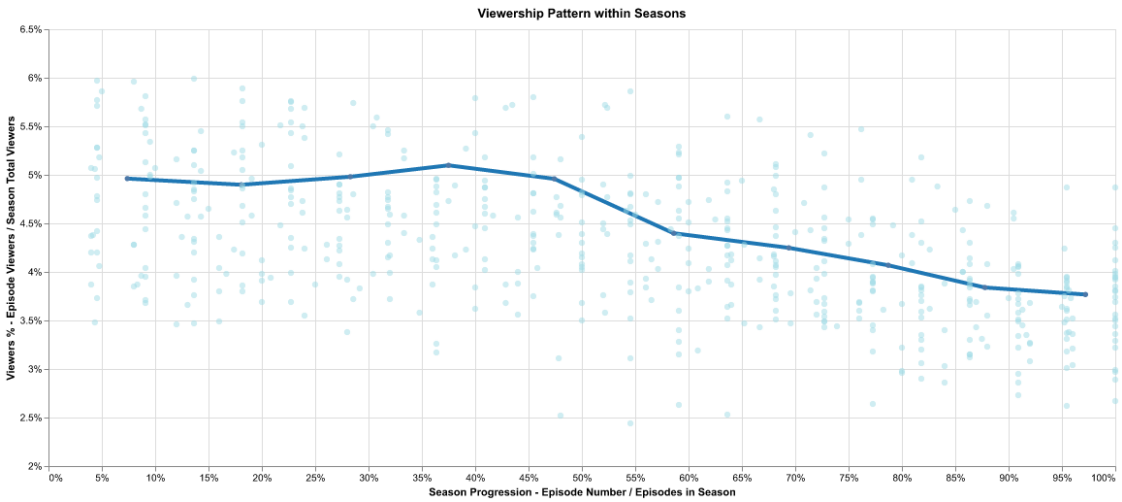
We also know that later seasons had more views than ealier ones, and since they are mixed in similar points of the X-axis, it was difficult to spot patterns. We addressed this by transforming viewers into a relative measure: the percentage of a season's total viewers.

To make the general trend easier to see, we added a tendency line calculated from the average of ten quantiles across the normalized X-axis.

The axes are based on newly calculated variables, which may not be obvious at first glance. We considered adding the formulas as a subtitle, but it used too much space, so we included them in the axis labels instead.

[19]:
```
make_plot_q5()
```

[19]:

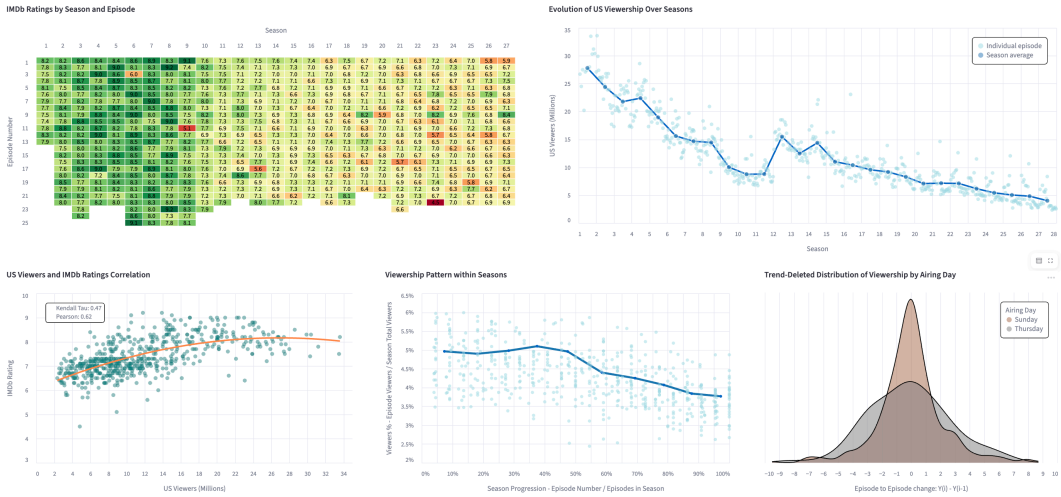


2.6 Final Visualization

The final visualization places Q1 and Q2 in the first row, while the second row contains Q3, Q5, and Q4. We made this layout choice because the first two plots need more horizontal space to be displayed correctly and be interpretable. The other three views can be interpreted well in a more compact way, so placing them on the second row balances intepretability and being space-efficient. The Dashboard has been desinged to fit in a laptop screen with minimal scroll needed, and with no scroll at all in a monitor screen.

The dashboard is designed to be read from left to right and top to bottom, following the order of the questions. We decided to change the order of Q4 and Q5, in order to display 3 similar charts one after the other. Q5 also provides more insights than Q4, so showing it first helps the reader extract the most relevant information earlier.

All viewership charts use blue tones, both for averages and individual episodes. Ratings are encoded with a green-to-red heatmap, which avoids overlap with the viewership palette. The correlation chart uses a distinct color because it represents the joint relationship between both variables. Finally, the density plot uses independent colors for weekdays, so the `day_aired` encoding is independent form the other ones. These have been selected to be distinguishable in light mode, so we explicitly set the Streamlit app to use it.



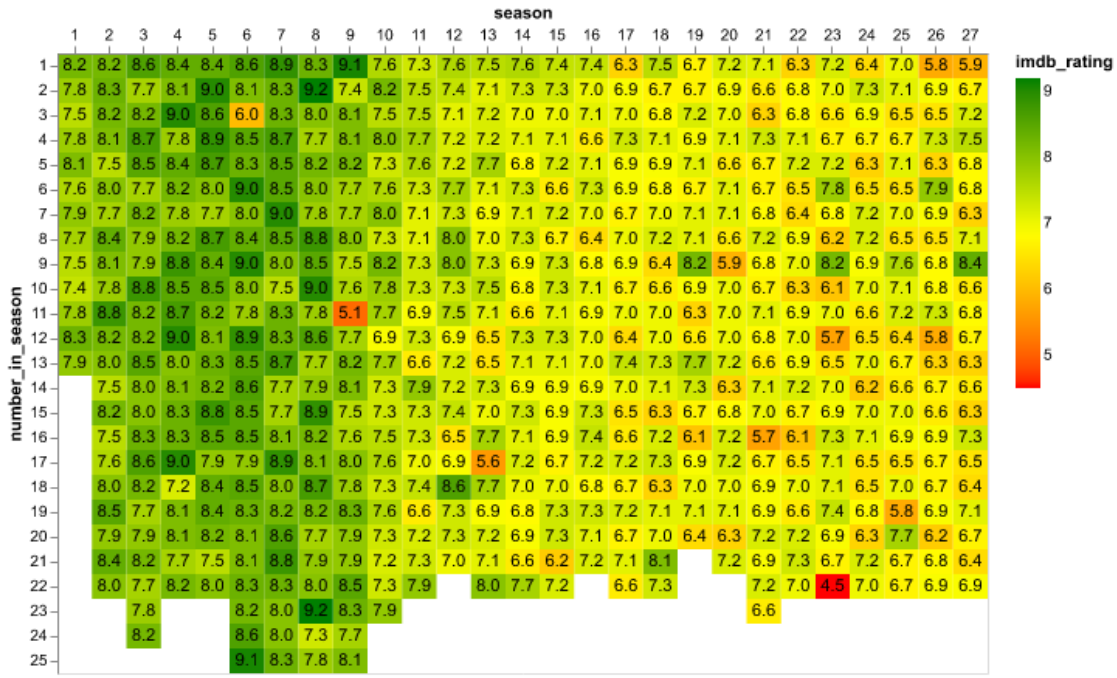
3 Initial Plot concepts

3.1 Q1

Heatmap with original color palette

[20]:
```
heatmap = alt.Chart(df_clean).mark_rect().encode(
    x = alt.X('season:N',axis=alt.Axis(orient='top',labelAngle=0)),
    y = alt.Y('number_in_season:N'),
    color = alt.Color('imdb_rating:Q',
        scale = alt.Scale(range=['red', 'yellow', 'green'])))

text = alt.Chart(df_clean).mark_text(baseline='middle').encode(
    x = alt.X('season:N'),
    y = alt.Y('number_in_season:N'),
    text = alt.Text('imdb_rating:Q', format=".1f")
)

chart = (heatmap + text).properties(width=600, height=400)
chart
```

[20]:



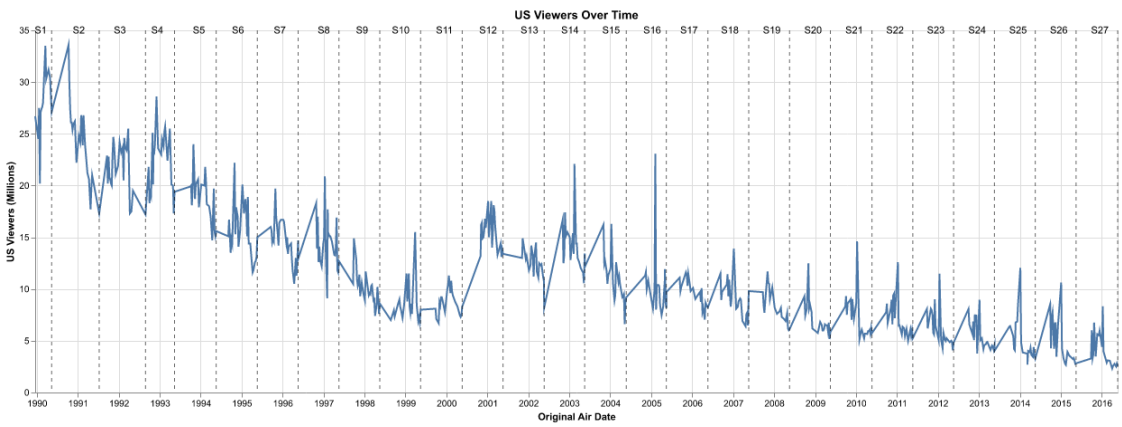
3.2 Q2

Line plot using all epsiodes instead of the average of the season

[21]:
```
line_chart = alt.Chart(df_clean).mark_line().encode(
    x=alt.X('original_air_date:T', title='Original Air Date'),
    y=alt.Y('us_viewers_in_millions:Q', title='US Viewers (Millions)')
).properties(
    title='US Viewers Over Time'
)

season_finals = df_clean[df_clean['is_last_episode'] == 1].copy()
limit_bounds = alt.Chart(season_finals).mark_rule(color='black', strokeDash=[4,␣
 ↪4], opacity=0.6).encode(
    x=alt.X('original_air_date:T'),
    tooltip=[alt.Tooltip('season:N', title='Season')]
 )

season_ranges = (
    df_clean.groupby('season', as_index=False).agg(
            start_date=('original_air_date', 'min'),
            end_date=('original_air_date', 'max')
        )
)
```

```
season_ranges['end_date'] = pd.to_datetime(season_ranges['end_date'])
season_ranges['start_date'] = pd.to_datetime(season_ranges['start_date'])
season_ranges['mid_date'] = season_ranges['start_date'] +␣
↪(season_ranges['end_date'] - season_ranges['start_date']) / 3
season_ranges['label_y'] = df_clean['us_viewers_in_millions'].max() * 1.03
season_ranges['season_label'] = 'S' + season_ranges['season'].astype(str)

season_labels = alt.Chart(season_ranges).
↪mark_text(dy=-4,color='black',fontSize=11).encode(
    x=alt.X('mid_date:T'),
    y=alt.Y('label_y:Q'),
    text=alt.Text('season_label:N', title='Season')
)

chart = line_chart + limit_bounds + season_labels
chart.properties(width=1200, height=400)
```

[21]:



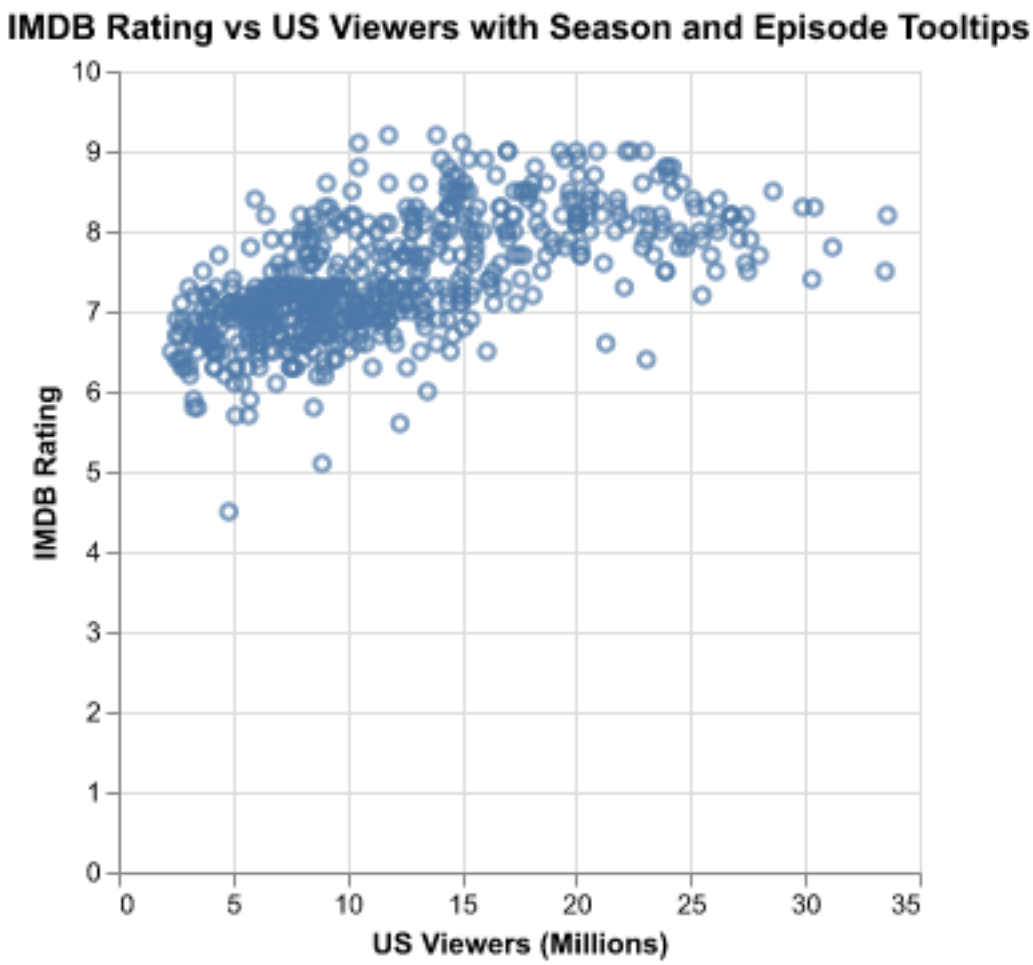
3.3 Q3

Initial scatter plot

[22]:

```
alt.Chart(df_clean).mark_point().encode(
    x=alt.X('us_viewers_in_millions:Q', title='US Viewers (Millions)'),
    y=alt.Y('imdb_rating:Q', title='IMDB Rating'),
    tooltip= ['season', 'number_in_season']
).properties(
    title='IMDB Rating vs US Viewers with Season and Episode Tooltips'
)
```

[22]:

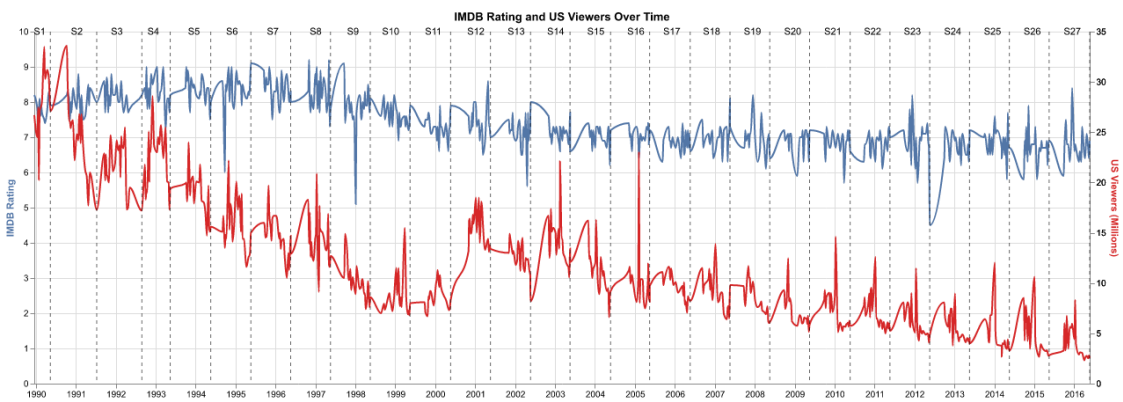


Dual axis for all epsidoes and ratings

[23]:
```
base = alt.Chart(df_clean).encode(
    alt.X('original_air_date:T', axis=alt.Axis(title=None)))

rating = base.mark_line(stroke='#5276A7', interpolate='monotone').encode(
    alt.Y('imdb_rating:Q', axis=alt.Axis(title='IMDB Rating',␣
 ↪titleColor='#5276A7')))


viewers = base.mark_line(stroke='#D62728', interpolate='monotone').encode(
    alt.Y('us_viewers_in_millions:Q', axis=alt.Axis(title='US Viewers␣
 ↪(Millions)', titleColor='#D62728')))

season_labels = alt.Chart(season_ranges).
 ↪mark_text(dy=-4,color='black',fontSize=11).encode(
    x=alt.X('mid_date:T'),
```

```
    y=alt.Y('label_y:Q',title=''),
    text=alt.Text('season_label:N', title='Season')
)

limit_bounds = alt.Chart(season_finals).mark_rule(color='black', strokeDash=[4,␣
↪4], opacity=0.6).encode(
    x=alt.X('original_air_date:T'),
    tooltip=[alt.Tooltip('season:N', title='Season')]
 )



alt.layer(rating, viewers, season_labels, limit_bounds).resolve_scale(y =␣
↪'independent').properties(width=1200, height=400, title='IMDB Rating and US␣
↪Viewers Over Time')
```

[23]:



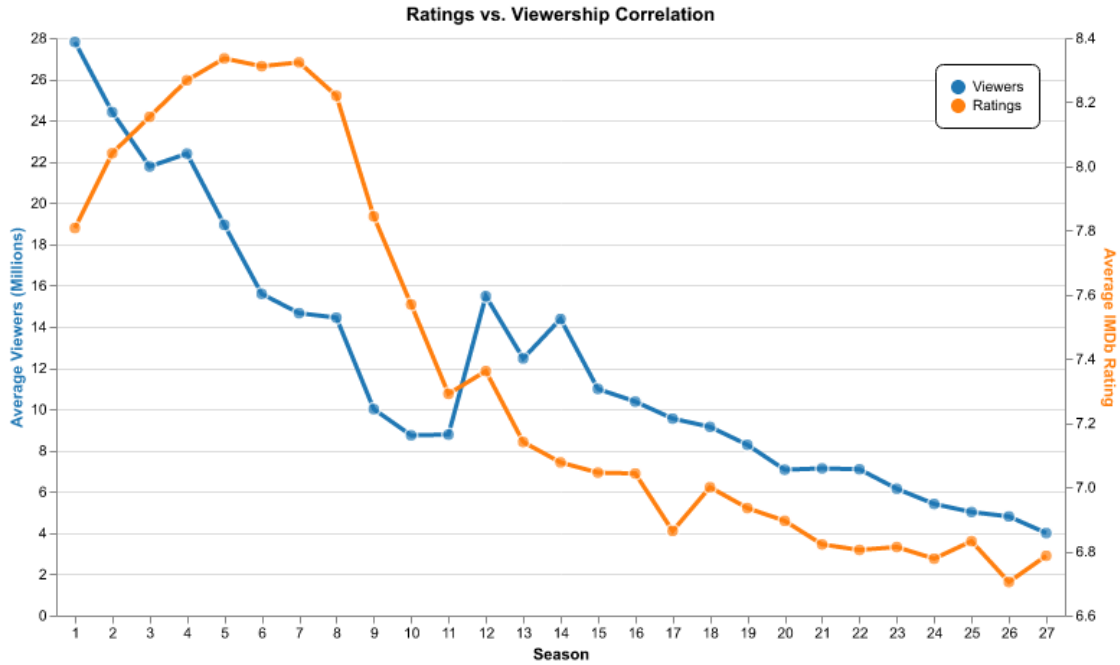
Double axis with the average of the seasons

[24]:

```
season_df = df_clean.groupby('season').agg({
        'us_viewers_in_millions': 'mean',
        'imdb_rating': 'mean'
    }).reset_index()

base = alt.Chart(season_df).encode(
        x=alt.X('season:O', title='Season', axis = alt.Axis(labelAngle=0))
    ).properties(title = "Ratings vs. Viewership Correlation", width=700,␣
↪height=400)

line_viewers = base.mark_line(color='#1f77b4', strokeWidth=3).encode(
        y=alt.Y('us_viewers_in_millions:Q', title='Average Viewers (Millions)',␣
↪axis=alt.Axis(titleColor='#1f77b4')))
```

```
point_viewers = base.mark_point(size=80, filled=True, stroke='white',␣
↪strokeWidth=1.5, color='#1f77b4').encode(
        y=alt.Y('us_viewers_in_millions:Q'),tooltip=[alt.Tooltip('season:O',␣
↪title='Season'),alt.Tooltip('us_viewers_in_millions:Q', title='Avg Viewers␣
↪(Millions)', format='.2f')]
    )


line_ratings = base.mark_line(color='#ff7f0e', strokeWidth=3).encode(
        y=alt.Y('imdb_rating:Q', title='Average IMDb Rating', axis=alt.
↪Axis(titleColor='#ff7f0e'), scale=alt.Scale(zero=False)))

point_ratings = base.mark_point(size=80, filled=True, stroke='white',␣
↪strokeWidth=1.5, color='#ff7f0e').encode(
        y=alt.Y('imdb_rating:Q',title ='').),tooltip=[alt.Tooltip('season:O',␣
↪title='Season'),alt.Tooltip('imdb_rating:Q', title='Avg IMDb Rating',␣
↪format='.2f')])

legend = alt.Chart(season_df).transform_fold(
        ['us_viewers_in_millions', 'imdb_rating'],
        as_=['metric', 'value']
    ).transform_calculate(
        series="datum.metric == 'us_viewers_in_millions' ? 'Average Viewers' :␣
↪'Average Ratings'"
    ).mark_point(size=1, opacity=0,filled =True ).encode(
        x=alt.X('season:O'),
        y=alt.Y('us_viewers_in_millions:Q',title=''),
        color=alt.Color(
            'series:N',
            title='',
            scale=alt.Scale(domain=['Viewers', 'Ratings'], range=['#1f77b4',␣
↪'#ff7f0e']),
            legend=alt.Legend(orient='top-right', strokeColor='black',␣
↪fillColor='white', cornerRadius=5, padding=10)
        )
    )
viewers_layer = alt.layer(line_viewers, point_viewers,legend)
ratings_layer = alt.layer(line_ratings, point_ratings)
chart2 = alt.layer(viewers_layer, ratings_layer).resolve_scale(y='independent').
↪properties(height=400)
chart2
```

[24]:



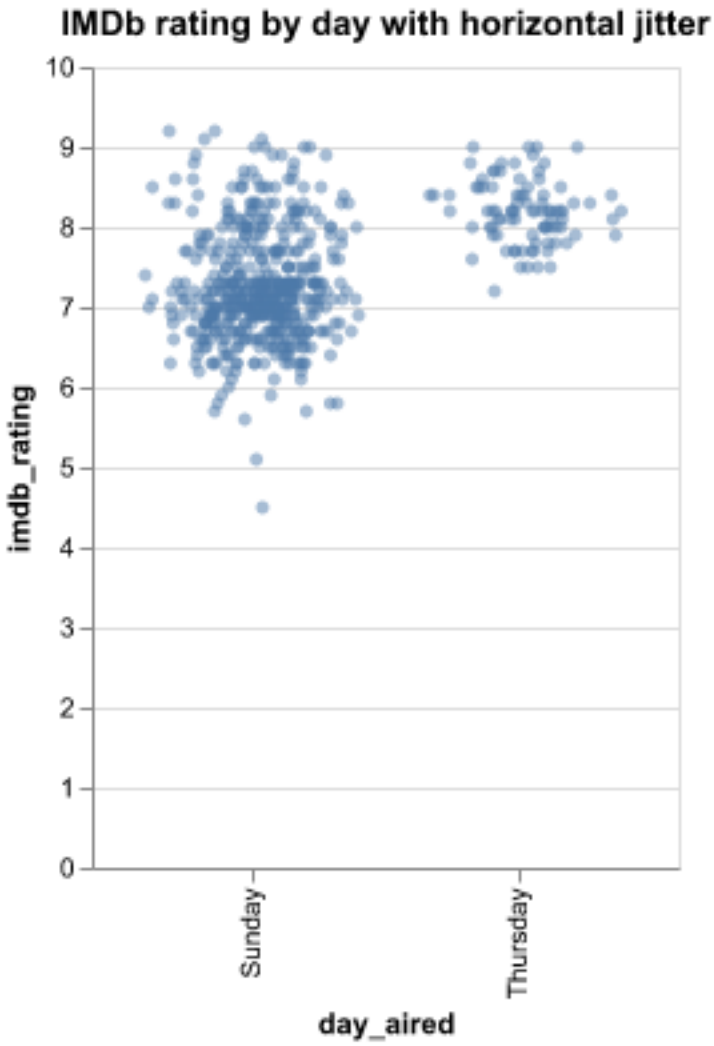
3.4 Q4

Initial jitter

source: https://altair-viz.github.io/gallery/strip_plot_jitter.html

[25]:
```
valid_airing = df_clean[df_clean['day_aired'].isin(['Sunday', 'Thursday'])]


alt.Chart(valid_airing).transform_calculate(jitter="(random() - 0.5) * 20").
 ↪mark_circle(size=25, opacity=0.5).encode(
        x=alt.X("day_aired:N"),
        y=alt.Y("imdb_rating:Q"),
        xOffset="jitter:Q"
    ).transform_calculate( jitter="sqrt(-2*log(random()))*cos(2*PI*random())").
 ↪properties(width=220, height=300, title="IMDb rating by day with horizontal␣
 ↪jitter")
```

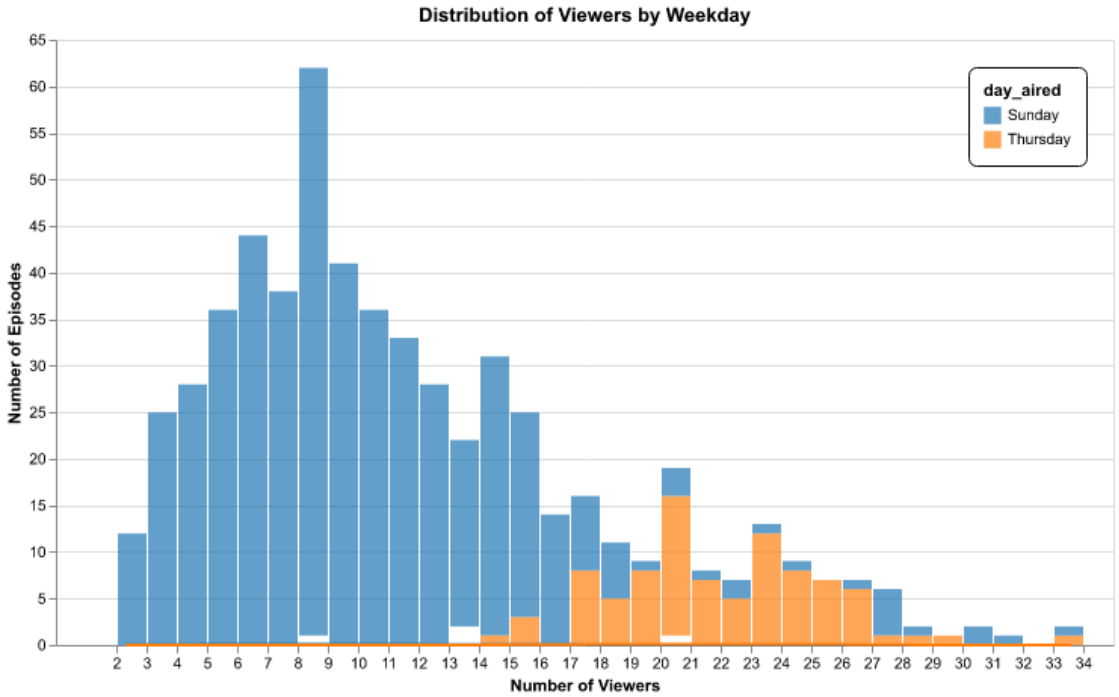
[25]:

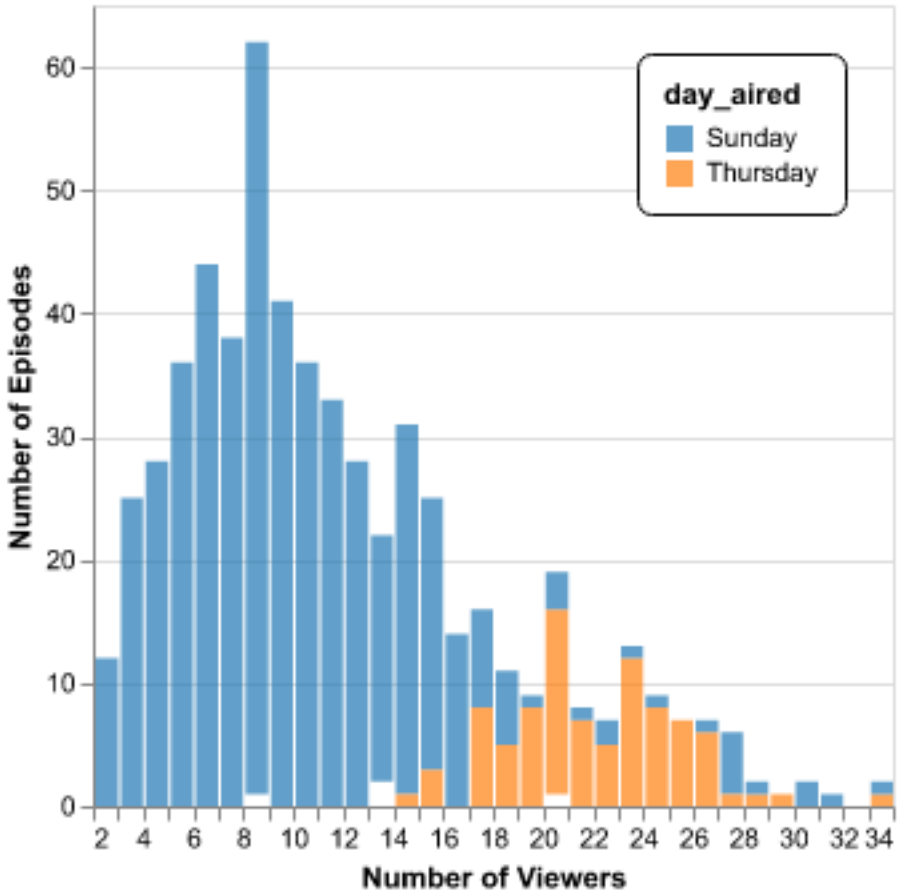


Initial density / histograms

[26]:
```
histogram = alt.Chart(df_clean).mark_bar(opacity=0.7).encode(
        x=alt.X(
            'us_viewers_in_millions',
            title='Number of Viewers',
            bin=alt.Bin(maxbins=60),
            axis=alt.Axis(format=',.0f'),
        ),
        y =alt.Y('count()', title='Number of Episodes'),
        tooltip=['day_aired', 'count()'],
        color=alt.Color('day_aired:N',
                        scale=alt.Scale(domain=['Sunday', 'Thursday'],␣
↪range=['#1f77b4', '#ff7f0e']),
```

```
                    legend=alt.Legend(orient='top-right', strokeColor='black',␣
↪fillColor='white', cornerRadius=5, padding=10))
    )

density = alt.Chart(df_clean).transform_density(
        'us_viewers_in_millions',
        as_=['us_viewers_in_millions', 'density'],
        groupby=['day_aired'],
        #bandwidth=1
    ).mark_line(size=2).encode(
        x=alt.X('us_viewers_in_millions', title='Number of Viewers', axis=alt.
↪Axis(format=',.0f')),
        y=alt.Y('density:Q', title=''),
        color=alt.Color(
            'day_aired:N',
            scale=alt.Scale(domain=['Sunday', 'Thursday'], range=['#1f77b4',␣
↪'#ff7f0e']),
            legend=None
        ),
        tooltip=['day_aired:N', 'us_viewers_in_millions:Q', 'density:Q']
    )

chart = (histogram + density).properties(
        width=700,
        height=400,
        title='Distribution of Viewers by Weekday'
    ).resolve_scale(
        y = 'shared'
    )

display(chart)
display(histogram)
```


Distribution of Viewers by Weekday
day_aired
Sunday
Thursday
Number of Episodes
Number of Viewers

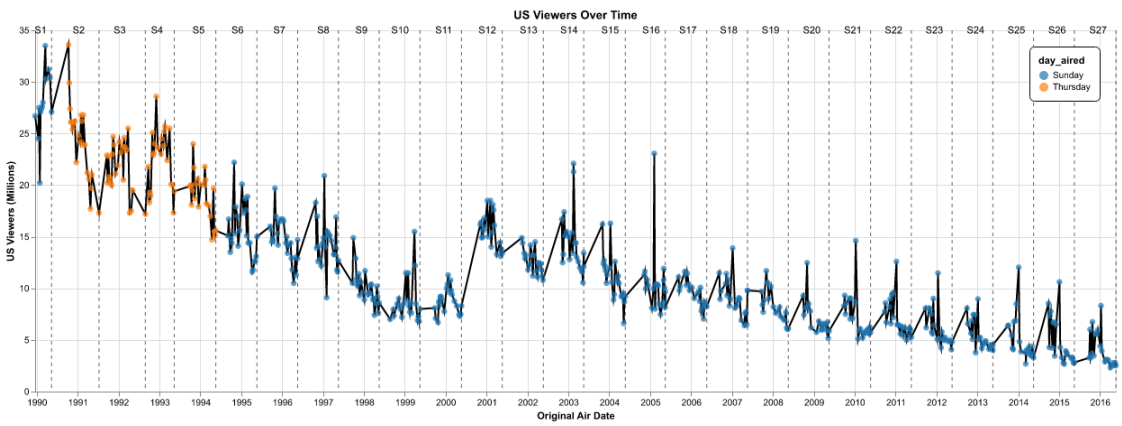

day_aired
Sunday
Thursday
Number of Episodes
Number of Viewers

Line plot by airing day

[27]:
```
df_clean['original_air_date'] = pd.to_datetime(df_clean['original_air_date'])
df_clean = df_clean.sort_values('original_air_date').copy()

base_line = alt.Chart(df_clean[df_clean['day_aired'].isin(['Sunday',␣
↪'Thursday'])]).mark_line(color='black').encode(
        x=alt.X('original_air_date:T', title='Original Air Date'),
        y=alt.Y('us_viewers_in_millions:Q', title='US Viewers (Millions)')
    )

highlighted_points = alt.Chart(df_clean[df_clean['day_aired'].isin(['Sunday',␣
↪'Thursday'])]).mark_point(size=42, filled=True).encode(
        x=alt.X('original_air_date:T', title='Original Air Date'),
        y=alt.Y('us_viewers_in_millions:Q', title='US Viewers (Millions)'),
        color=alt.Color(
            'day_aired:N',
            scale=alt.Scale(domain=['Sunday', 'Thursday'], range=['#1f77b4',␣
↪'#ff7f0e']),
            legend=alt.Legend(orient='top-right', strokeColor='black',␣
↪fillColor='white', cornerRadius=5, padding=10)
        ),
        tooltip=['original_air_date:T', 'number_in_season:Q',␣
↪'us_viewers_in_millions:Q', 'day_group:N', 'season:N']
    )

line_chart = (base_line + highlighted_points).properties(
        title='US Viewers Over Time'
    )

season_finals = df_clean[df_clean['is_last_episode'] == 1].copy()
limit_bounds = alt.Chart(season_finals).mark_rule(color='black', strokeDash=[4,␣
↪4], opacity=0.6).encode(
        x=alt.X('original_air_date:T'),
        tooltip=[alt.Tooltip('season:N', title='Season')]
    )

season_ranges = (
        df_clean.groupby('season', as_index=False)
        .agg(
            start_date=('original_air_date', 'min'),
            end_date=('original_air_date', 'max')
        )
    )
```

```
season_ranges['end_date'] = pd.to_datetime(season_ranges['end_date'])
season_ranges['start_date'] = pd.to_datetime(season_ranges['start_date'])
season_ranges['mid_date'] = season_ranges['start_date'] +␣
↪(season_ranges['end_date'] - season_ranges['start_date']) / 3
season_ranges['label_y'] = df_clean['us_viewers_in_millions'].max() * 1.03
season_ranges['season_label'] = 'S' + season_ranges['season'].astype(str)

season_labels = alt.Chart(season_ranges).mark_text(
        dy=-4,
        color='black',
        fontSize=11
    ).encode(
        x=alt.X('mid_date:T'),
        y=alt.Y('label_y:Q'),
        text=alt.Text('season_label:N', title='Season')
    )

chart = line_chart + limit_bounds + season_labels
chart.properties(width=1200, height=400,)
```

[27]:



3.5 Q5

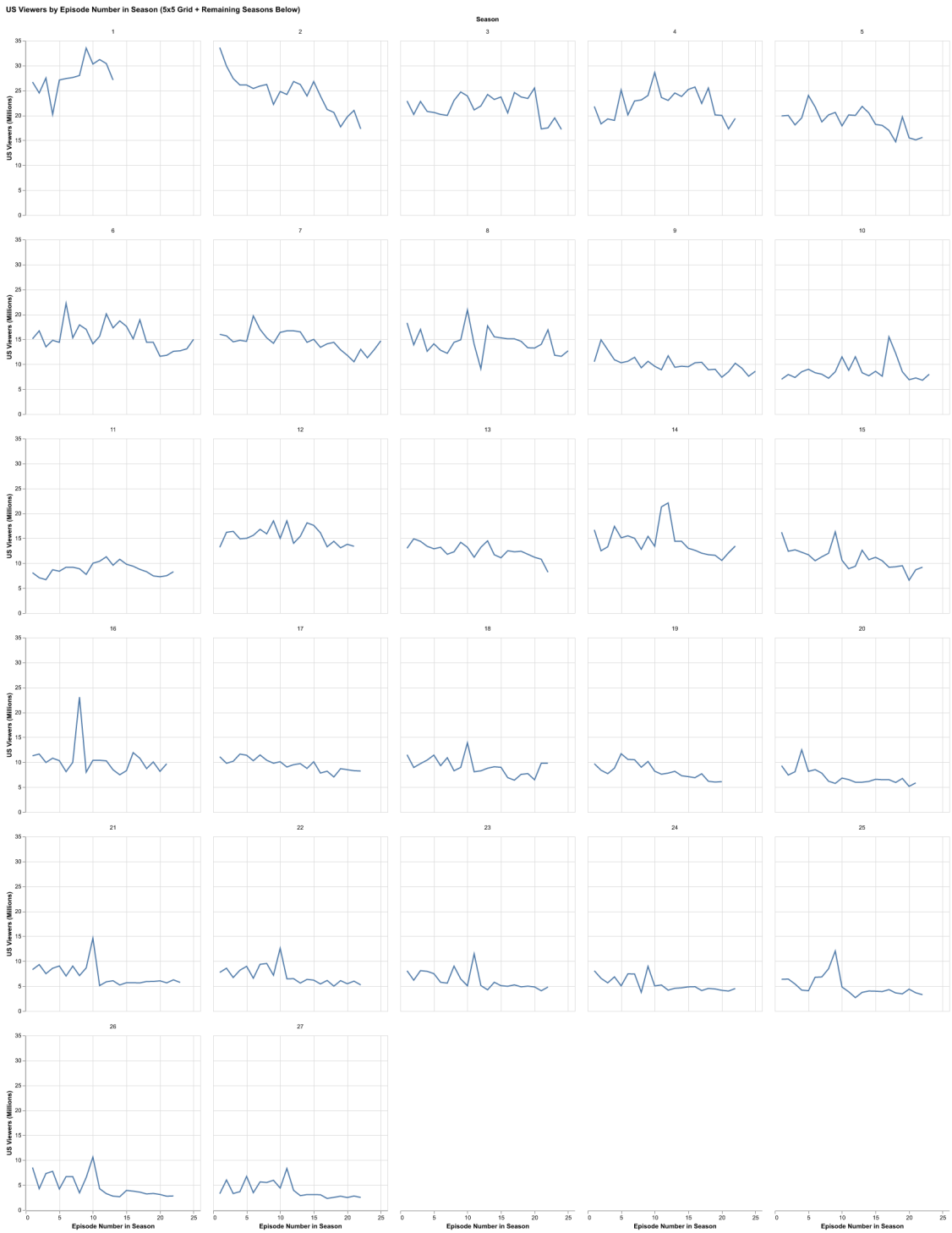
Initial multiple views

[28]:

```
alt.Chart(df_clean).mark_line().encode(
        x=alt.X("number_in_season:Q", title="Episode Number in Season"),
        y=alt.Y("us_viewers_in_millions:Q", title="US Viewers (Millions)")
    ).facet(
        facet=alt.Facet("season:N", title="Season"),
        columns=5
    ).resolve_scale(
        x="shared",
        y="shared").properties(
```

```
        title="US Viewers by Episode Number in Season (5x5 Grid + Remaining␣
↪Seasons Below)",
    )
```

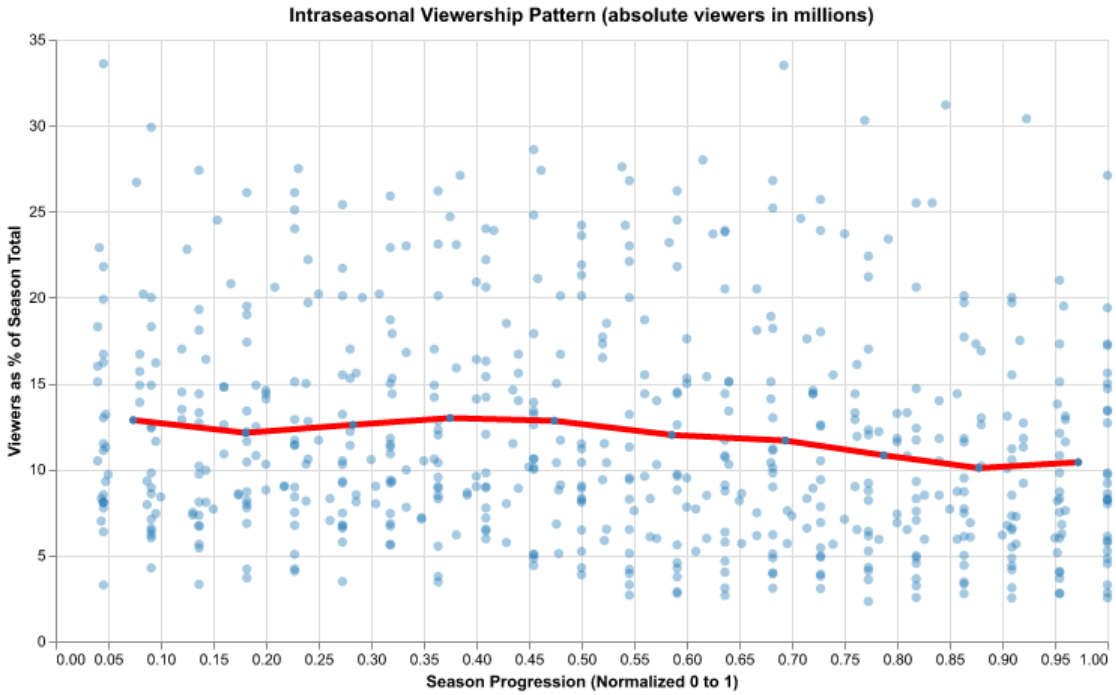
[28]:



Initial plot with absolute x axis

[29]:
```
df_clean['number_of_episode_normalized'] = df_clean['number_in_season'] /␣
↪df_clean.groupby('season')['number_in_season'].transform('max')
df_clean['number_of_episode_normalized_quantiles'] = pd.
↪qcut(df_clean['number_of_episode_normalized'], q=10, labels=False)


scatter = alt.Chart(df_clean).mark_circle(opacity=0.4, size=40,␣
↪color='#1f77b4').encode(
        x=alt.X('number_of_episode_normalized:Q', title='Season Progression␣
↪(Normalized 0 to 1)'),
        y=alt.Y('us_viewers_in_millions:Q', title='Viewers as % of Season␣
↪Total'),
        tooltip=['season', 'number_in_season','us_viewers_in_millions']
    )

trend_line = alt.Chart(df_clean).mark_line(color='red', strokeWidth=4,␣
↪point=True).encode(
        x=alt.X('mean(number_of_episode_normalized):Q'),
        y=alt.Y('mean(us_viewers_in_millions):Q'),
        tooltip=[
            alt.Tooltip('number_of_episode_normalized_quantiles:O',␣
↪title='Season Phase (Quantile)'),
            alt.Tooltip('mean(us_viewers_in_millions):Q', title='Average␣
↪viewers (millions)', format='.2f')
        ]
    )

final_chart_absolute = (scatter + trend_line).properties(
        width=700,
        height=400,
        title="Intraseasonal Viewership Pattern (absolute viewers in millions)"
    )
final_chart_absolute
```

[29]:


Intraseasonal Viewership Pattern (absolute viewers in millions)
Viewers as % of Season Total
Season Progression (Normalized 0 to 1)
0
5
10
15
20
25
30
35
0.00 0.05 0.10 0.15 0.20 0.25 0.30 0.35 0.40 0.45 0.50 0.55 0.60 0.65 0.70 0.75 0.80 0.85 0.90 0.95 1.00